

Quantitative Finance with Multiple Trading Algorithms

Luka Saarivirta

October 3, 2026

Contents

1	Introduction	3
1.1	Research Objective	3
1.2	Research Question	3
1.3	Research Scope	3
2	Data	4
2.1	Data Source	4
2.2	Feature Selection & Preprocessing	4
2.3	Data Limitations	4
3	Methodology	5
3.1	Backtesting Engine	5
3.2	Portfolio Simulation	5
3.2.1	Valuation Mechanics	5
3.2.2	Position Sizing & Signal Execution	6
3.2.3	Execution Assumptions	6
3.3	Performance Metrics	6
3.3.1	Sharpe Ratio	6
3.3.2	Maximum Drawdown	7
4	Trading Strategies	8
4.1	Buy and Hold (Benchmark)	8
4.2	Momentum Strategy	8
4.3	Moving Average Strategy	8
4.4	KNN-Filtered Momentum Strategy	9
4.5	Standard Deviation Moving Average Mean Reversion	10
4.6	XGBoost	10
4.7	Neural Network	11

5	Results	13
5.1	Buy and Hold Performance	13
5.2	Momentum Strategy Performance	13
5.3	Moving Average Performance	14
5.4	KNN-Filtered Momentum Performance	15
5.5	Standard Deviation Moving Average Mean Reversion Performance	17
5.6	XGBoost Performance	18
5.7	Neural Network Performance	19
6	Discussion	22
6.1	Interpretation of Results	22
6.2	Hypothesis Testing	22
6.3	Methodological Limitations	22
6.4	Sources of Bias	23
6.5	Future Research Directions	23
7	Conclusion	24
8	Declaration of AI Assistance	25
9	References	26

1 Introduction

This paper investigates the performance of systematic trading algorithms applied to historical S&P 500 equity data.

1.1 Research Objective

The primary objective of this research is to evaluate how strategy efficacy varies across assets with distinct *risk profiles*, *volatility metrics*, and *sector classifications*.

1.2 Research Question

To what extent do quantitative trading strategies (such as *trend-following momentum* and *moving average crossovers*) demonstrate consistent risk-adjusted returns when applied across different equity market sectors compared to a *passive benchmark*?

1.3 Research Scope

To evaluate the intrinsic effectiveness of each algorithmic baseline, this study enforces a strict non-optimization protocol: all strategies are implemented using their default, uncalibrated parameter configurations without post-hoc tuning.

The empirical evaluation is conducted across three equities chosen to represent distinct economic sectors, market dynamics, and volatility profiles:

1. **AAPL (Apple Inc.):** A mega-cap technology asset representing secular growth, high momentum, and exceptional liquidity.
2. **CAT (Caterpillar Inc.):** A large-cap industrial asset heavily exposed to macroeconomic cycles and commodity trends.
3. **KO (The Coca-Cola Company):** A defensive consumer staples asset characterized by low market beta, lower volatility, and steady dividend-driven returns.

2 Data

This section outlines the dataset origin, feature selection, and preprocessing steps executed prior to backtesting.

2.1 Data Source

Historical price data was sourced from the S&P 500 stock market repository provided by Kaggle (`rprkh15/sp500-stock-prices`).

2.2 Feature Selection & Preprocessing

For each asset in the target scope (AAPL, CAT, KO), time series data was structured using the *trading timestamp* as the primary indexing variable. The primary feature used for execution modeling is the *daily opening price* (P_t).

2.3 Data Limitations

The historical data series excludes *live market updates* and *extended trading hours*. Additionally, using raw opening prices excludes the impact of *intraday high-low spreads*, *corporate actions*, *dividends*, and *execution slippage*.

3 Methodology

This section formalizes the theoretical framework, backtesting engine parameters, and performance criteria.

3.1 Backtesting Engine

To execute the market simulations across an expanding suite of trading models, a custom, modular backtesting engine was implemented in Python. The engine employs an extensible *strategy-registry pattern* that decouples strategy design from execution logic, allowing seamless scaling to additional quantitative algorithms without requiring modifications to the core engine architecture.

The backtesting workflow is organized into four core functional components:

- **Input Validation & State Initialization:** Ingests historical time-series market data (`pd.DataFrame`) and initializes base state variables, including *starting cash capital* ($C_0 = \$1,000$) and *initial asset holdings* ($N_0 = 0$).
- **Dynamic Strategy Dispatcher:** Replaces rigid conditional routing with a dynamic lookup mechanism. Strategy modules register their execution entry points independently, enabling the dispatcher to call arbitrary strategy algorithms dynamically based on standardized parameter signatures.
- **Standardized Interface Contract:** Enforces a uniform input-output contract across all strategy implementations. Every strategy receives identical market state feeds and returns a standardized historical ledger tracking portfolio states.
- **Performance Reporting & Output Generation:** Yields a continuous time-series `DataFrame` containing daily tracking of cash reserves (`Cash`), market execution price (`Open`), position exposure (`Shares`), and total equity (`Portfolio Value`), while computing aggregate summary metrics at completion.

3.2 Portfolio Simulation

The portfolio simulation model governs the discrete-time accounting mechanics applied across the trading evaluation period. Let $t \in \{0, 1, 2, \dots, T\}$ represent discrete daily trading intervals. The portfolio state at day t is fully characterized by the tuple (C_t, N_t, V_t) , where C_t represents *cash reserves*, N_t denotes the *number of held equity shares*, and V_t is the *total portfolio valuation*.

3.2.1 Valuation Mechanics

At any trading day t , the *total portfolio value* V_t is computed as the sum of uninvested capital and the mark-to-market value of the open equity position evaluated at the execution price P_t :

$$V_t = C_t + N_t \cdot P_t$$

3.2.2 Position Sizing & Signal Execution

Trading actions are governed by a *discrete strategy signal* $S_t \in \{-1, 0, 1\}$, where $S_t = 1$ indicates a *long entry request*, $S_t = 0$ signals an *exit/cash stance*, and $S_t = -1$ denotes a *short position* (restricted in this baseline setup).

Execution occurs at the market open price P_t according to the following dynamic updating rules:

1. **Capital Allocation (Long Signal $S_t = 1$):** If $S_t = 1$ and $N_{t-1} = 0$, available capital C_{t-1} is deployed to purchase the maximum number of whole shares:

$$\Delta N_t = \left\lfloor \frac{C_{t-1}}{P_t} \right\rfloor, \quad N_t = N_{t-1} + \Delta N_t$$

$$C_t = C_{t-1} - (\Delta N_t \cdot P_t)$$

2. **Position Liquidation (Exit Signal $S_t = 0$):** If $S_t = 0$ and $N_{t-1} > 0$, the open position is fully liquidated at price P_t :

$$C_t = C_{t-1} + (N_{t-1} \cdot P_t), \quad N_t = 0$$

3. **Inaction / Hold Stance:** If $S_t = S_{t-1}$, no trades execute, and state parameters persist ($N_t = N_{t-1}$, $C_t = C_{t-1}$).

3.2.3 Execution Assumptions

The simulation operates under *friction-free execution assumptions* for the baseline experiment:

- Zero transaction costs and zero exchange fees ($g = 0$).
- Zero execution slippage ($P_{\text{executed}} = P_t$).
- Fractional share purchases are prohibited ($\Delta N_t \in \mathbb{Z}^+$).

3.3 Performance Metrics

To evaluate risk-adjusted performance across backtested strategies, two primary quantitative metrics are tracked:

3.3.1 Sharpe Ratio

The annualized *Sharpe Ratio* measures return per unit of total risk. Let $R_t = \frac{V_t - V_{t-1}}{V_{t-1}}$ represent daily portfolio returns, $\bar{R}$ the average daily return, and σ_d the sample standard deviation of daily returns. Assuming a risk-free rate $R_f = 0$ and 252 annual trading days, the annualized Sharpe Ratio is formalized as:

$$\text{Sharpe Ratio} = \sqrt{252} \cdot \frac{\bar{R} - R_f}{\sigma_d}$$

3.3.2 Maximum Drawdown

Maximum Drawdown (MDD) quantifies peak-to-trough downside volatility, representing the maximum cumulative loss experienced by an investor over the period T :

$$\text{MDD} = \max_{\tau \in [0, T]} \left(\frac{\max_{t \in [0, \tau]} V_t - V_\tau}{\max_{t \in [0, \tau]} V_t} \right)$$

4 Trading Strategies

This section details the formal mechanics and decision rules of each evaluated strategy.

4.1 Buy and Hold (Benchmark)

The *Buy and Hold* strategy serves as the baseline benchmark (B). At time $t = 0$, the maximum number of whole shares attainable with initial capital C_0 is purchased at the initial open price P_0 :

$$N_{\text{shares}} = \left\lfloor \frac{C_0}{P_0} \right\rfloor$$

The position remains static across the backtest duration and is liquidated at the final available opening price P_T .

4.2 Momentum Strategy

The *Momentum strategy* operates under the assumption of short-term price trend persistence. It calculates an n -day simple return using opening prices:

$$r_t(n) = \frac{P_t}{P_{t-n}} - 1$$

where P_t represents the opening price on trading day t , and P_{t-n} represents the opening price n trading days prior.

The trading signal $S_t \in \{0, 1\}$ is generated according to the following decision rules:

- **Long Signal** ($S_t = 1$): If $r_t(n) > 0$, allocate available capital to purchase whole shares.
- **Exit/Cash Signal** ($S_t = 0$): If $r_t(n) \leq 0$, liquidate active positions at P_t and hold cash.

For the baseline experiments, the *lookback window* is parameterized to $n = 5$ trading days.

4.3 Moving Average Strategy

The *Moving Average Strategy* smooths price fluctuations over a fixed lookback period m to identify baseline trends:

$$\text{SMA}_t(m) = \frac{1}{m} \sum_{i=0}^{m-1} P_{t-i}$$

The executive trading signal $S_t \in \{0, 1\}$ is generated by evaluating the current open execution price P_t relative to $\text{SMA}_t(m)$:

- **Long Signal** ($S_t = 1$): If $P_t > \text{SMA}_t(m)$, allocate capital to open equities.
- **Exit/Cash Signal** ($S_t = 0$): If $P_t \leq \text{SMA}_t(m)$, liquidate positions and hold uninvested capital.

4.4 KNN-Filtered Momentum Strategy

The *K-Nearest Neighbors* (KNN) strategy extends the *Momentum Strategy* by using historical market states to filter momentum signals. When the momentum condition is satisfied, the model evaluates whether similar historical market states were followed by positive returns.

The momentum return is calculated using:

$$r_t(n) = \frac{P_t}{P_{t-n}} - 1$$

When the momentum signal is active, a normalized feature vector is constructed:

$$X_t = \begin{bmatrix} r_t(1) \\ r_t(5) \\ \sigma_t \end{bmatrix}$$

where $r_t(1)$ and $r_t(5)$ represent the one-day and five-day returns, respectively, and σ_t represents the measured price volatility.

The Euclidean distance between the current market state X_t and a historical state X_h is calculated as:

$$d(X_t, X_h) = \sqrt{\sum_{j=1}^3 (X_{t,j} - X_{h,j})^2}$$

The K historical observations with the smallest distances are selected as the nearest neighbors. Each neighbor is classified according to whether its subsequent forward return was positive:

$$y_h = \begin{cases} 1, & r_h(n_f) > 0 \\ 0, & r_h(n_f) \leq 0 \end{cases}$$

where n_f represents the forward-return evaluation period.

The fraction of nearest neighbors associated with positive forward returns is then calculated as:

$$\hat{p}_t = \frac{1}{K} \sum_{h \in N_K(X_t)} y_h$$

where $N_K(X_t)$ denotes the set of K nearest historical observations.

The final trading signal $S_t \in \{0, 1\}$ is generated according to the following decision rules:

- **Long Signal** ($S_t = 1$): If the momentum condition is satisfied and $\hat{p}_t > 0.5$, allocate available capital to purchase whole shares.
- **Exit/Cash Signal** ($S_t = 0$): If the momentum condition is not satisfied or $\hat{p}_t \leq 0.5$, liquidate active positions and hold cash.

For the baseline experiments, the momentum lookback window is parameterized to $n = 5$ trading days, while the KNN parameters K and the forward-return period n_f are fixed according to the experimental configuration.

4.5 Standard Deviation Moving Average Mean Reversion

The *Standard Deviation Moving Average Mean Reversion* strategy operates under the assumption that large statistical deviations of the price from its moving average will eventually be followed by a movement back toward the average. It calculates the normalized variation (Z -score) of the current price from the moving average:

$$V_t = \frac{P_t - \text{MA}_t(n)}{\sigma_t(n)}$$

where P_t represents the current price, $\text{MA}_t(n)$ represents the moving average calculated over the previous n trading days, and $\sigma_t(n)$ represents the sample standard deviation calculated over the same lookback window n .

- **Long Signal** ($S_t = 1$): If $V_t \leq -T$, allocate available capital to purchase as many whole shares as possible.
- **Cash Signal** ($S_t = 0$): If $V_t \geq T$, liquidate any active position and hold the resulting capital as cash.
- **Hold Signal** ($S_t = 2$): If $-T < V_t < T$, take no trading action and maintain the current portfolio position.

4.6 XGBoost

The *XGBoost* strategy uses a gradient-boosted decision tree classifier to predict the direction of the next trading day's opening price. Unlike the rule-based strategies, the trading signal is determined by a supervised machine learning model trained on historical market data.

Two features are calculated for each trading day. The first is the 20-day normalized deviation of the opening price from its moving average:

$$Z_t = \frac{P_t - \text{MA}_t(20)}{\sigma_t(20)}$$

where P_t represents the opening price, $\text{MA}_t(20)$ represents the 20-day moving average, and $\sigma_t(20)$ represents the sample standard deviation over the same lookback window.

The second feature is the one-day percentage return:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

The feature vector used by the classifier is therefore:

$$\mathbf{x}_t = [Z_t \quad R_t]$$

The target variable indicates whether the following opening price is higher than the current opening price:

$$y_t = \begin{cases} 1 & \text{if } P_{t+1} > P_t \\ 0 & \text{otherwise} \end{cases}$$

The XGBoost classifier consists of an ensemble of decision trees whose predictions are combined to estimate the probability of an upward price movement:

$$\hat{p}_t = P(y_t = 1 \mid \mathbf{x}_t)$$

The model is trained exclusively on observations preceding the beginning of the out-of-sample backtest period. This prevents observations from the backtest period from being used during model training.

During the backtest, the predicted probability is converted into a trading signal using a confidence threshold of $C = 0.52$:

$$S_t = \begin{cases} 1 & \text{if } \hat{p}_t \geq C \\ 0 & \text{if } \hat{p}_t < C \end{cases}$$

where $S_t = 1$ represents a long position and $S_t = 0$ represents a cash position.

When a long signal is generated while no shares are held, the strategy purchases the maximum number of whole shares affordable with the available capital:

$$N_t = \left\lfloor \frac{C_t}{P_t} \right\rfloor$$

where C_t represents the available capital and N_t represents the number of shares purchased.

When a cash signal is generated while holding shares, the entire position is liquidated and the resulting capital is held as cash.

4.7 Neural Network

The neural network uses ten input features: High, Open, Low, Close, 2-day moving average, 5-day moving average, 10-day moving average, 20-day moving average, 5-day return, and 10-day return.

The target variable indicates whether the next opening price is higher than the current opening price:

$$y_t = \begin{cases} 1 & \text{if } \text{Open}_{t+1} > \text{Open}_t \\ 0 & \text{otherwise} \end{cases}$$

The data is divided chronologically into 70% training data and 30% testing data. The features are normalized using the mean and standard deviation of the training data:

$$X_{\text{normalized}} = \frac{X - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

The same training-set statistics are applied to the testing data.

The neural network consists of three fully connected layers with the following architecture:

$$10 \rightarrow 10 \rightarrow 20 \rightarrow 1$$

The first two layers use the ReLU activation function:

$$\text{ReLU}(x) = \max(0, x)$$

The final layer produces a single logit. The model is trained for 1,000 epochs using binary cross-entropy loss with logits and the AdamW optimizer. The learning rate is 0.001 and the weight decay is 0.01.

After training, the logits are converted into probabilities using the sigmoid function:

$$P(y = 1) = \frac{1}{1 + e^{-z}}$$

where z represents the output logit of the neural network.

The predicted probability is converted into a trading signal using a threshold of 0.5:

$$S_t = \begin{cases} 1 & \text{if } P(y = 1) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

where $S_t = 1$ represents a long position and $S_t = 0$ represents a cash position.

When a long signal is generated, the strategy attempts to hold the maximum number of whole shares affordable using the total portfolio value. When a cash signal is generated, all held shares are sold.

The portfolio value is calculated as:

$$V_t = C_t + N_t P_t$$

where C_t is the available cash, N_t is the number of shares held, and P_t is the current opening price.

5 Results

This section presents the results of all algorithms compared with the Buy and Hold strategy.

5.1 Buy and Hold Performance

The *Buy and Hold benchmark* performance is depicted across all strategy comparison plots.

5.2 Momentum Strategy Performance

The empirical backtest results indicate that the simple momentum strategy exhibits superior trend-capture capabilities when applied to cyclical assets (e.g., CAT) compared to highly volatile or range-bound assets.

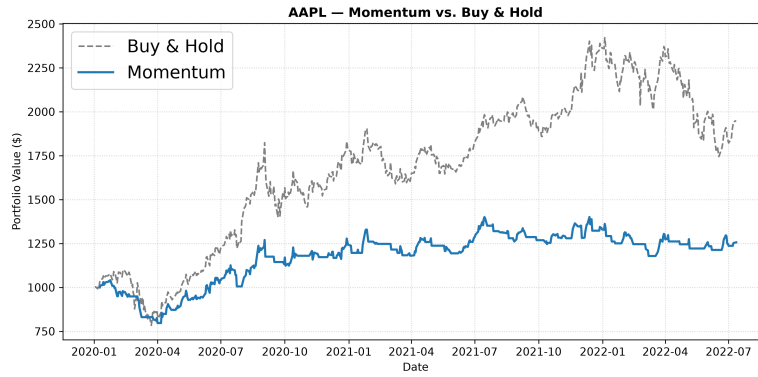


Figure 1: Performance comparison on AAPL (High-Volatility Growth Sector).

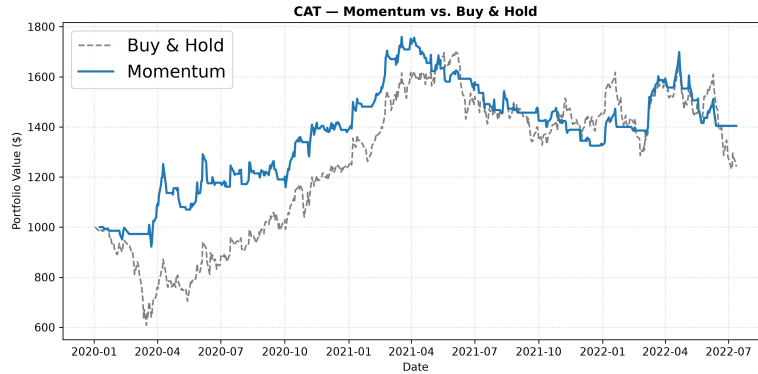


Figure 2: Performance comparison on CAT (Cyclical Industrial Sector).

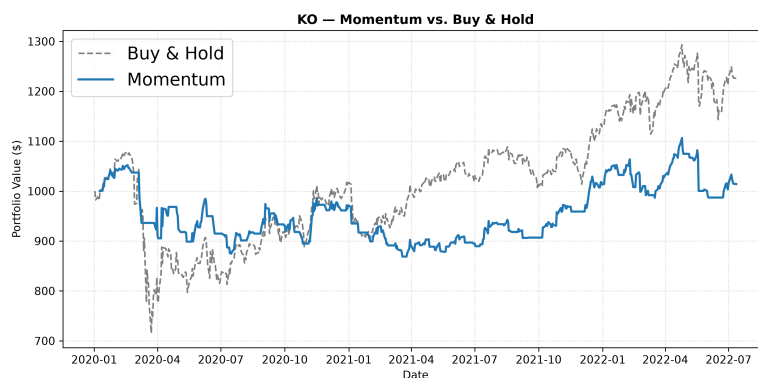


Figure 3: Performance comparison on KO (Defensive Consumer Staples Sector).

5.3 Moving Average Performance

The empirical backtest results for the Moving Average strategy demonstrate reduced portfolio turnover and drawdowns during prolonged structural shifts.

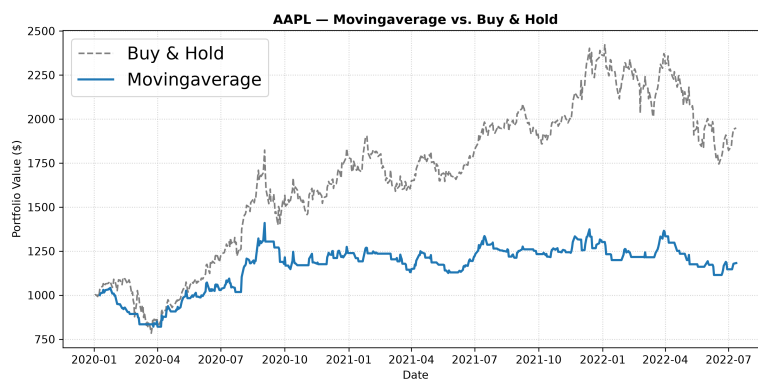


Figure 4: Performance comparison on AAPL (High-Volatility Growth Sector).

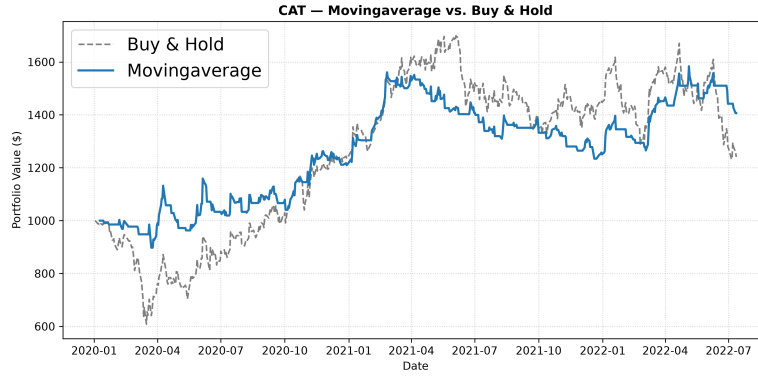


Figure 5: Performance comparison on CAT (Cyclical Industrial Sector).

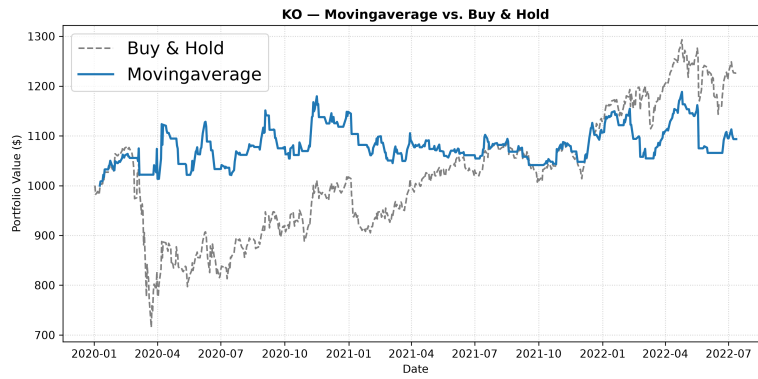


Figure 6: Performance comparison on KO (Defensive Consumer Staples Sector).

5.4 KNN-Filtered Momentum Performance

The KNN-filtered momentum strategy does not excel in any one of our tests. The strategy exhibits a relatively horizontal linear trend near its lower region.

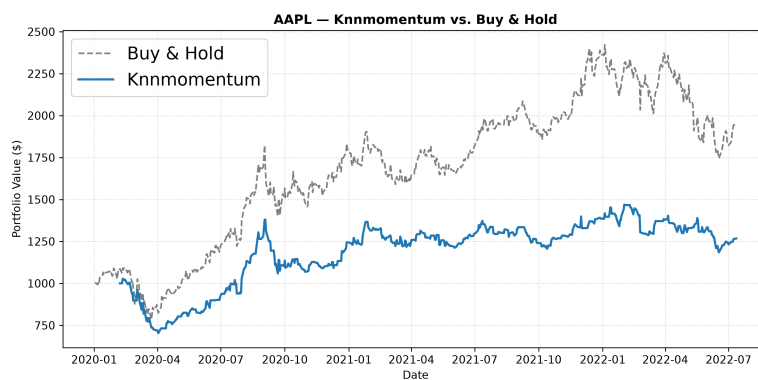


Figure 7: Performance comparison on AAPL (High-Volatility Growth Sector).

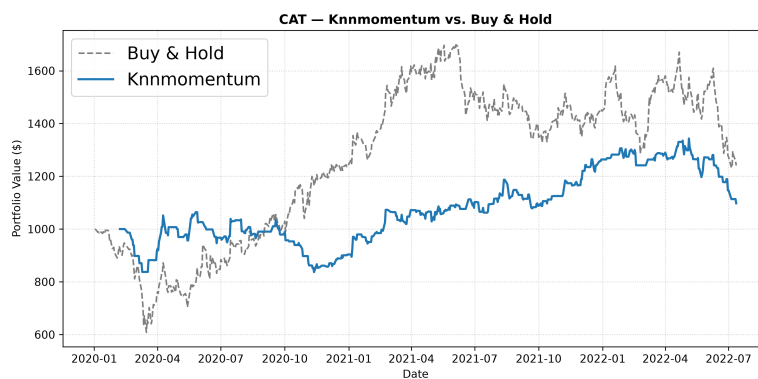


Figure 8: Performance comparison on CAT (Cyclical Industrial Sector).

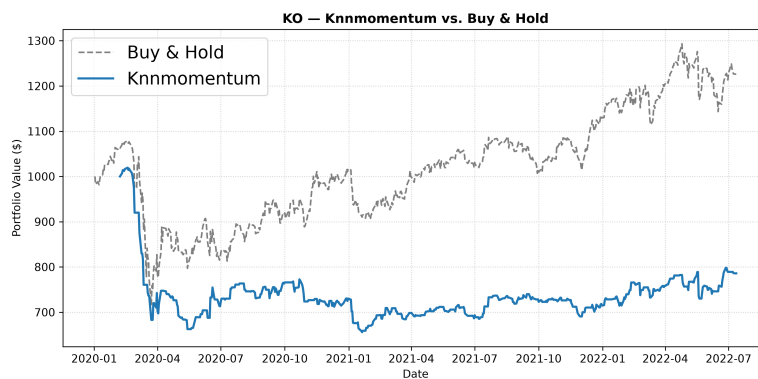


Figure 9: Performance comparison on KO (Defensive Consumer Staples Sector).

5.5 Standard Deviation Moving Average Mean Reversion Performance

The standard deviation moving average mean reversion strategy severely underperforms the buy-and-hold baseline across all tested equities. By waiting for rare, extreme statistical deviations to trigger entries and exits, the strategy spends prolonged periods sitting in cash, thereby missing the multi-year compound growth of long-term upward trends.

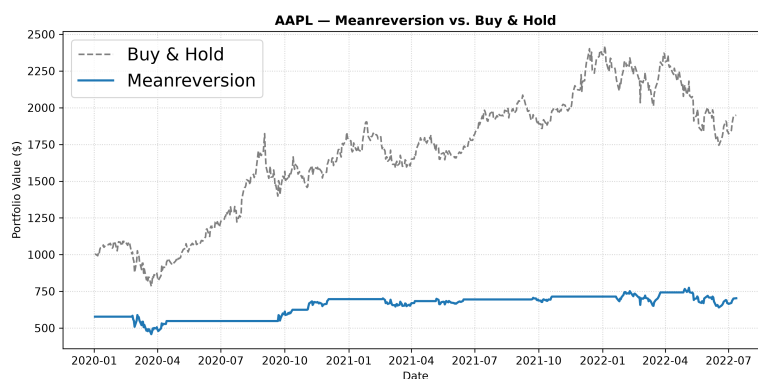


Figure 10: Performance comparison on AAPL (High-Volatility Growth Sector).

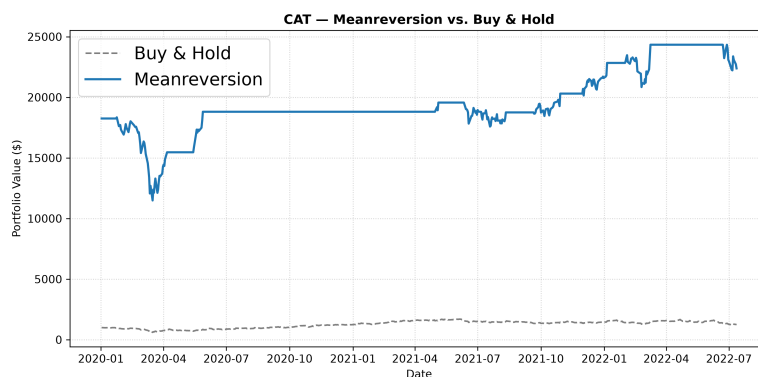


Figure 11: Performance comparison on CAT (Cyclical Industrial Sector).

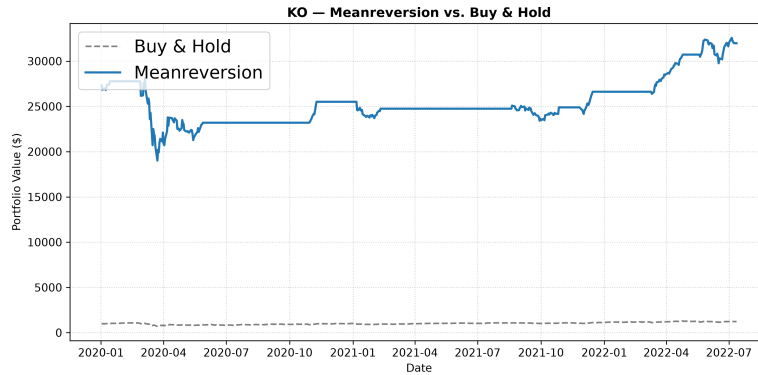


Figure 12: Performance comparison on KO (Defensive Consumer Staples Sector).

5.6 XGBoost Performance

The XGBoost strategy showed moderate and relatively consistent performance on AAPL and CAT. Its performance remained relatively stable throughout the testing period, staying around the middle of the graph. In contrast, the performance on KO was considerably lower. After an initial decline, it remained at a low level for most of the testing period, indicating that the strategy was less effective for KO.

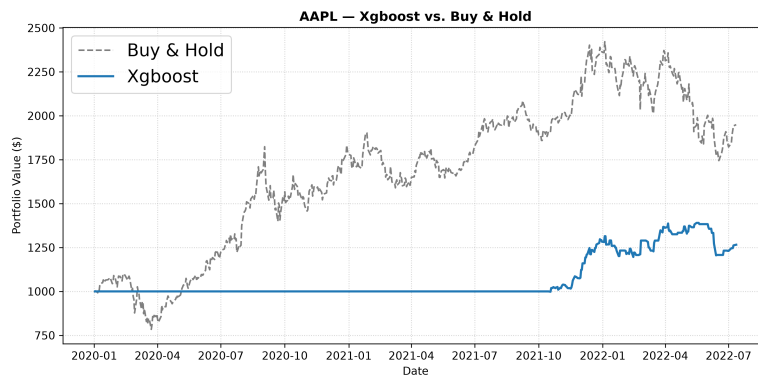


Figure 13: Performance comparison on AAPL (High-Volatility Growth Sector).

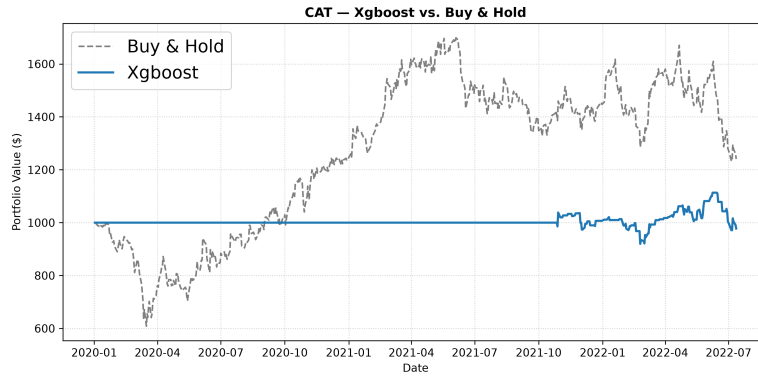


Figure 14: Performance comparison on CAT (Cyclical Industrial Sector).

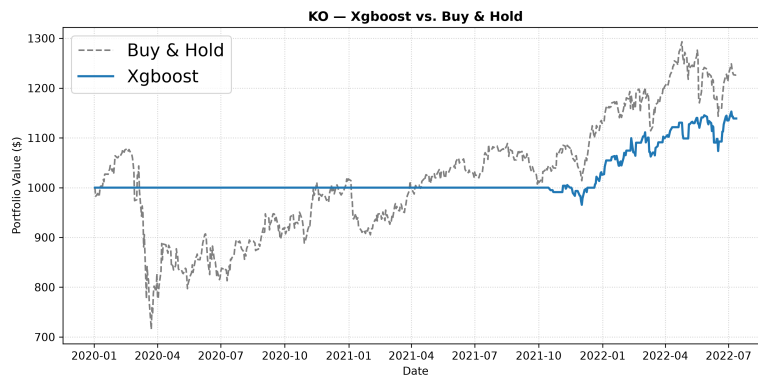


Figure 15: Performance comparison on KO (Defensive Consumer Staples Sector).

5.7 Neural Network Performance

The multi-layer perceptron neural network strategy displayed mixed performance across sectors, demonstrating sensitivity to signal noise in out-of-sample data.

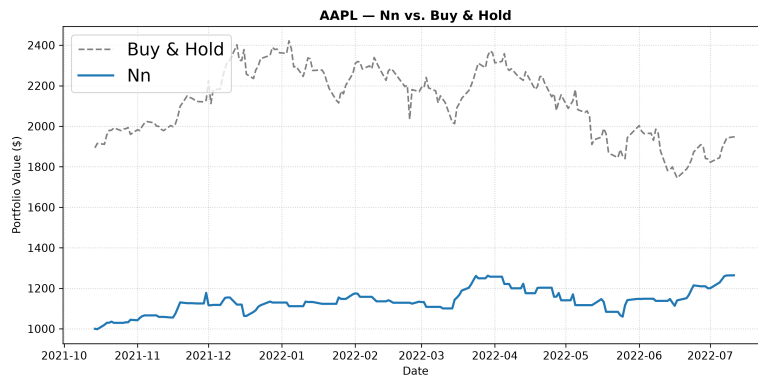


Figure 16: Performance comparison on AAPL (High-Volatility Growth Sector).

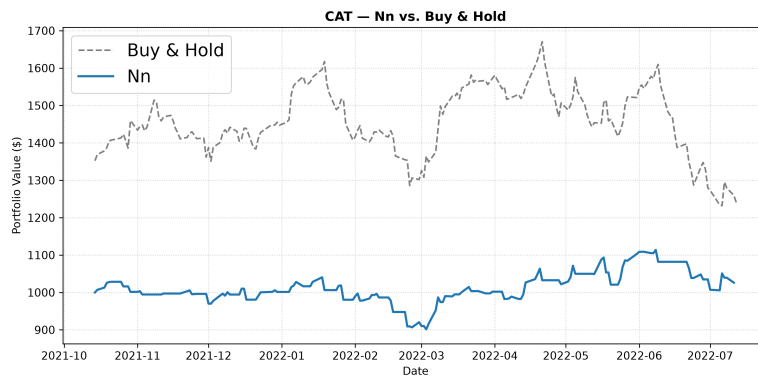


Figure 17: Performance comparison on CAT (Cyclical Industrial Sector).

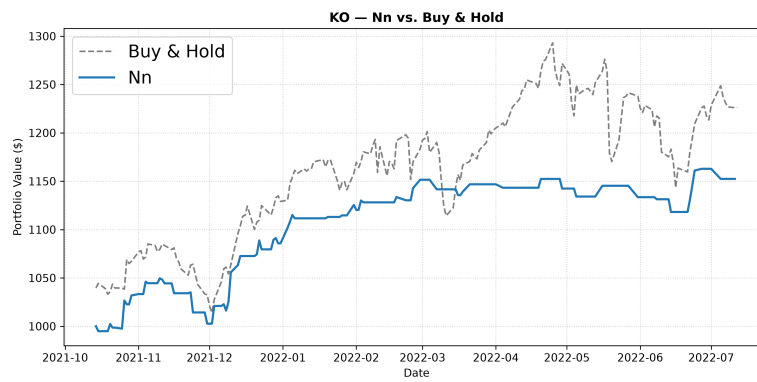


Figure 18: Performance comparison on KO (Defensive Consumer Staples Sector).

6 Discussion

6.1 Interpretation of Results

- **Rule-Based Benchmarks vs. Complex Models:** Simple trend-following models (Momentum and Moving Average) captured broad directional trends effectively during strong multi-year market expansions.
- **KNN-Filtered Momentum Underperformed:** The *KNN-Filtered Momentum* strategy fell well short of expectations. While the concept sounds promising on paper, adding extra layers of complexity introduces unnecessary friction—causing the model to hold back from buying or selling when the obvious, profitable move was to take the trade.
- **Mean Reversion Inefficiency:** The Standard Deviation Moving Average Mean Reversion strategy suffered from extreme cash drag by enforcing strict deviation thresholds ($V_t \leq -T$), missing long-term upside momentum in secularly growing equities.
- **Machine Learning & Neural Networks:** Tree-based models (XGBoost) and Neural Networks exhibited moderate performance on higher-volatility growth stocks (AAPL, CAT) but underperformed on lower-beta assets (KO), highlighting high vulnerability to out-of-sample distributional shift.

6.2 Hypothesis Testing

- H_0 (**Null Hypothesis**): The algorithmic trading strategies (Momentum, Moving Average, KNN-Filtered Momentum, Mean Reversion, XGBoost, and Neural Network) do not generate statistically significant excess risk-adjusted returns (Sharpe ratio) relative to the Buy & Hold benchmark.
- H_1 (**Alternative Hypothesis**): The systematic trading strategies outperform the Buy & Hold baseline on a risk-adjusted basis.
- **Outcome:** Based on the backtested portfolio values and risk metrics, simple benchmark strategies often maintained strong performance, whereas higher-complexity models (e.g., KNN-Filtered Momentum) failed to consistently beat Buy & Hold, leading to a failure to reject H_0 across all complex strategies.

6.3 Methodological Limitations

- **Fixed Asset Universe:** The study evaluated a limited subset of equities (e.g., AAPL, CAT, KO), which may introduce asset-specific behavior that does not generalize across all market conditions or sectors.

- **Backtest Horizon:** Testing over a specific historical window (e.g., starting 2020) exposes the results to regime-specific dynamics such as COVID-19 volatility and market recovery phases.
- **Parameter Fixedness:** Moving average windows (m) and hyperparameter choices for machine learning models were fixed rather than dynamically recalibrated via rolling cross-validation.

6.4 Sources of Bias

- **Look-Ahead Bias:** Ensured y_t signals use information available strictly at time t , though signal execution at $Open_{t+1}$ must assume zero execution delay or slippage.
- **Survivorship Bias:** Selecting high-profile, continuously traded equities over the sample period inherently selects historical market winners.
- **Overfitting Risk:** Machine learning strategies like XGBoost, KNN, and Neural Networks risk overfitting to historical feature distributions without guaranteeing out-of-sample stability.

6.5 Future Research Directions

- **Dynamic Parameter Optimization:** Incorporate rolling walk-forward optimization for parameter selection to adapt to changing volatility regimes.
- **Transaction & Friction Modeling:** Introduce realistic bid-ask spread friction, market impact costs, and variable order execution delays to evaluate post-cost performance.
- **Expanded Multi-Asset Universe:** Apply the strategies across broader asset classes (e.g., indices, commodities, forex) to evaluate cross-asset robustness.

7 Conclusion

This paper systematically evaluated a diverse suite of quantitative trading algorithms—ranging from classical rule-based indicators to supervised machine learning architectures—against a passive Buy and Hold benchmark across three distinct market equities (AAPL, CAT, and KO).

The empirical results demonstrate that increased algorithmic complexity does not consistently yield superior risk-adjusted returns in uncalibrated settings. Simple trend-following strategies effectively captured structural market movements, whereas complex filtering models like KNN-Filtered Momentum and Neural Networks suffered from trade friction and cash drag. Consequently, passive buy-and-hold investing remains a formidable benchmark against uncalibrated quantitative algorithms during prolonged upward trending regimes.

8 Declaration of AI Assistance

During the preparation of this paper, ChatGPT and Gemini were used solely for grammatical editing and text polishing. The author created, structured, and formulated all core arguments, concepts, and technical content, and takes full responsibility for the manuscript.

9 References

References

- [1] R. Prajapati, “S&P 500 Stock Prices Dataset,” Kaggle Dataset Repository, 2023. [Online]. Available: <https://www.kaggle.com/datasets/rprkh15/sp500-stock-prices>.
- [2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [3] E. Fix and J. L. Hodges, “Discriminatory analysis: Nonparametric discrimination: Consistency properties,” *International Statistical Review / Revue Internationale de Statistique*, vol. 57, no. 3, pp. 238–247, 1989.
- [4] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [5] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [6] W. F. Sharpe, “The Sharpe ratio,” *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, 1994.
- [7] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS 32)*, 2019, pp. 8024–8035.
- [8] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] W. McKinney, “Data structures for statistical computing in Python,” in *Proceedings of the 9th Python in Science Conference (SciPy)*, 2010, pp. 56–61.